

PLANO: Máquina de Vendas como Projeto Deployável Completo

Cada máquina = um repositório independente com FRONTEND + BACKEND + DB + APIs + AGENTS + DEPLOY

PRIMEIRA AÇÃO após aprovação: Salvar este plano em melhorias/plano-maquina-deployavel.md + .pdf

STACK CONFIRMADA

Camada	Tecnologia	Justificativa
Frontend	Next.js 14 (App Router)	SSR, API routes, Tailwind, deploy Vercel
Backend	FastAPI (Python)	Async, OpenAPI automático, combina com scripts existentes
Database	SQLite	Zero config, arquivo único, MVP até 10k leads
Deploy	Docker + Vercel	Flexível (VPS ou cloud)
Criação	Máquina completa	Full-stack: frontend + backend + DB + scripts + deploy

1. CONCEITO

Cada máquina de vendas gerada a partir de uma obra é um **projeto full-stack autônomo**:

```
marketing/maquinas/{slug}/
├── frontend/           # Next.js – landing pages, captura, vendas
├── backend/            # FastAPI – APIs, webhooks, automações
├── database/           # SQLite/Postgres – leads, métricas, funis
├── .claude/            # AGENTS, SKILLS, COMMANDS da máquina
├── .mcp.json           # MCPS específicos (email, pagamento, CRM)
├── config/             # Produtos, funis, personas, roteamento LLM
├── scripts/            # Automação (lead hunter, monitor, auto-correct)
├── templates/          # HTML emails, posts, stories
├── docker-compose.yml  # Deploy local ou VPS
├── vercel.json          # Deploy frontend (Vercel)
├── AGENTS.md           # Orquestrador da máquina
└── CLAUDE.md           # Regras e config do agente
```

— SPEC.md	# Spec completa da máquina
— README.md	# Documentação + manual de deploy

2. STACK DE CADA MÁQUINA

Frontend (Next.js 14+ / App Router)

- / — Página de venda do produto core
- /captura — Página de captura com lead magnet
- /obrigado — Página de agradecimento + tripwire
- /checkout — Página de checkout (integração pagamento)
- /admin — Dashboard de métricas (protegido)
- /api/lead — API route para captura de lead
- /api/webhook — Webhook de pagamento

Backend (FastAPI)

- POST /api/leads — Cadastro de lead
- POST /api/leads/qualificar — Scoring automático
- GET /api/leads — Listagem com filtros
- POST /api/emails/enviar — Disparo de sequência
- POST /api/dm/enviar — Disparo de DM
- GET /api/funil/metricas — Métricas do funil
- POST /api/funil/auto-correct — Auto-correção
- POST /api/webhook/pagamento — Webhook Stripe/Kiwify
- GET /api/health — Health check

Database (SQLite para MVP, Postgres para produção)

- leads — Email, nome, score, fonte, estágio
 - interacoes — Cliques, aberturas, páginas visitadas
 - vendas — Produto, valor, data, status
 - campanhas — Nome, status, métricas
 - emails_enviados — Lead, template, status, data
 - metricas_diarias — Conversão por etapa, data
-

3. ESTRUTURA DE DIRETÓRIOS (detalhada)

marketing/maquinas/{slug}/

—	frontend/		# Next.js App Router
	—	app/	
		—	layout.tsx
		—	page.tsx
		—	captura/
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—
			—

├─ checkout/	
│ └─ page.tsx	# Checkout (/checkout)
├─ admin/	
│ └─ layout.tsx	# Layout admin
│ └─ page.tsx	# Dashboard
│ └─ leads/	
│ │ └─ page.tsx	# Gestão de leads
│ └─ emails/	
│ │ └─ page.tsx	# Gestão de e-mails
│ └─ metrics/	
│ │ └─ page.tsx	# Métricas detalhadas
└─ api/	
│ └─ lead/	
│ │ └─ route.ts	# API: cadastro lead
│ └─ webhook/	
│ │ └─ route.ts	# API: webhook pagamento
│ └─ health/	
│ │ └─ route.ts	# API: health check
├─ components/	
│ └─ Hero.tsx	# Seção hero da página de venda
│ └─ ValueStack.tsx	# Stack de valor
│ └─ Testimonials.tsx	# Depoimentos
│ └─ PricingCard.tsx	# Card de preço
│ └─ Guarantee.tsx	# Seção de garantia
│ └─ LeadForm.tsx	# Formulário de captura
│ └─ EmailSequence.tsx	# Preview de e-mails
│ └─ MetricsChart.tsx	# Gráfico de métricas
├─ lib/	
│ └─ api.ts	# Cliente API backend
│ └─ analytics.ts	# Tracking de eventos
├─ public/	
│ └─ artes/	# Artes geradas pela fábrica
│ └─ og-image.png	# Open Graph image
├─ tailwind.config.ts	
├─ next.config.ts	
├─ package.json	
└─ tsconfig.json	
├─ backend/	# FastAPI
│ └─ app/	
│ │ └─ main.py	# App FastAPI
│ │ └─ models/	
│ │ │ └─ lead.py	# Modelo Lead
│ │ │ └─ interacao.py	# Modelo Interação
│ │ │ └─ venda.py	# Modelo Venda
│ │ │ └─ campanha.py	# Modelo Campanha
│ │ └─ routers/	
│ │ │ └─ leads.py	# Rotas de leads
│ │ │ └─ emails.py	# Rotas de e-mails

```

├── ──┬── funil.py          # Rotas de funil
│     └── webhooks.py      # Rotas de webhooks
│   ──┬── services/
│     ├── lead_service.py  # Lógica de leads
│     ├── email_service.py # Lógica de e-mails
│     ├── scoring_service.py # Scoring de leads
│     ├── metricas_service.py # Métricas do funil
│     └── auto_correct.py   # Auto-correção
│   ──┬── database/
│     ├── connection.py    # Conexão DB
│     └── migrations.py    # Schema/migrations
│   ──┬── config.py         # Configurações
│   ──┬── requirements.txt   # FastAPI, uvicorn, sqlite
│   ──┬── Dockerfile
├── database/
│   ├── schema.sql         # Schema completo
│   ├── seed.sql           # Dados iniciais
│   └── backups/           # Backups automáticos
├── .claude/               # Agentes e skills da máquina
│   ├── agents/
│   │   ├── subagente-copywriter.md
│   │   ├── subagente-designer-artes.md
│   │   ├── subagente-narrador-audio.md
│   │   ├── subagente-criador-video.md
│   │   ├── subagente-qualificador-leads.md
│   │   ├── subagente-analista-funil.md
│   │   ├── subagente-campanha-email.md
│   │   └── subagente-gestor-trafego.md
│   ├── skills/
│   │   ├── criar-pagina-venda/SKILL.md
│   │   ├── criar-pagina-captura/SKILL.md
│   │   ├── criar-sequencia-emails/SKILL.md
│   │   ├── criar-sequencia-dm/SKILL.md
│   │   ├── criar-artes/SKILL.md
│   │   ├── criar-audio/SKILL.md
│   │   ├── criar-videos/SKILL.md
│   │   ├── monitorar-funil/SKILL.md
│   │   └── auto-correct/SKILL.md
│   └── commands/
│       ├── status.md      # /status
│       ├── monitorar.md   # /monitorar
│       ├── corrigir.md    # /corrigir
│       └── escalar.md     # /escalar
├── scripts/
│   ├── lead_hunter.py     # Busca leads Instagram
│   └── email_sender.py    # Disparo de e-mails

```

		funnel_monitor.py	# Monitoramento 24/7
		auto_correct.py	# Auto-correção
		metrics_collector.py	# Coleta de métricas
		deploy.sh	# Script de deploy
		templates/	
		emails/	# Templates de e-mail
		boas_vindas.html	
		nutricao_01.html	
		venda_01.html	
		reativacao_01.html	
		posts/	# Templates de posts
		post_feed.html	
		story.html	
		dm/	# Templates de DM
		dm_primeiro_contato.txt	
		dm_followup.txt	
		config/	
		produtos.json	# Escada de valor
		funis.json	# Config de funis
		personas.json	# ICPs
		canais.json	# Config de canais
		email.json	# Config provedor e-mail
		pagamento.json	# Config gateway pagamento
		roteamento_modelos.json	# Roteamento LLM
		subagentes.json	# Registry de subagentes
		.mcp.json	# MCPs da máquina
		docker-compose.yml	# Deploy
		vercel.json	# Deploy frontend
		.env.example	# Variáveis de ambiente
		AGENTS.md	# Orquestrador
		CLAUDE.md	# Regras do agente
		SPEC.md	# Spec completa
		README.md	# Documentação + deploy

4. SCRIPT PRINCIPAL: `criar-maquina-vendas.py`

O script orquestrador que gera o projeto completo:

Entrada: slug da obra + configurações do operador

Saída: projeto completo em `marketing/maquinas/{slug}/`

Etapas:

1. Lê obra em `output/{slug}/`
2. Descobre modelos disponíveis (`descobrir_modelos.py`)
3. Gera estrutura de diretórios
4. Gera frontend (Next.js) a partir de templates

5. Gera backend (FastAPI) a partir de templates
 6. Gera schema do banco
 7. Gera .mcp.json com MCPS necessários
 8. Gera AGENTS.md e CLAUDE.md da máquina
 9. Gera SPEC.md da máquina
 10. Gera config/*.json
 11. Gera scripts/ de automação
 12. Gera docker-compose.yml e vercel.json
 13. Copia templates de e-mails/posts/DM
 14. Roda subagentes em paralelo (copy, artes, etc.)
 15. Gera README.md com manual de deploy
-

5. MCPS DE CADA MÁQUINA

```
{
  "mcpServers": {
    "db_state": {
      "command": "node",
      "args": ["mcp-servers/sqlite/index.js", "database/maquina.db"]
    },
    "file_writer": {
      "command": "node",
      "args": ["mcp-servers/filesystem/index.js", "."]
    },
    "mcp_email": {
      "command": "node",
      "args": ["mcp-servers/email/index.js"],
      "env": { "SENDGRID_API_KEY": "${SENDGRID_API_KEY}" }
    },
    "mcp_payments": {
      "command": "node",
      "args": ["mcp-servers/payments/index.js"],
      "env": { "STRIPE_SECRET_KEY": "${STRIPE_SECRET_KEY}" }
    },
    "mcp_instagram": {
      "command": "node",
      "args": ["mcp-servers/instagram/index.js"],
      "env": { "INSTAGRAM_TOKEN": "${INSTAGRAM_TOKEN}" }
    }
  }
}
```

6. AGENTS.md DE CADA MÁQUINA

MÁQUINA DE VENDAS: {título da obra}

Regras

- Operação 24/7 autônoma
- Auto-correção quando conversão < threshold

- Escala automática quando ROAS > 2x
- LGPD compliance obrigatório

Squad

copywriter → designer-artes → qualificador-leads →
campanha-email → analista-funil → auto-correct

Comandos

- /status – dashboard da máquina
 - /monitorar – métricas em tempo real
 - /corrigir – auto-correção
 - /escalar – aumentar budget/alcance
-

7. DEPLOY

Opção 1: Docker (VPS)

```
# docker-compose.yml
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
  backend:
    build: ./backend
    ports: ["8000:8000"]
    volumes: [".:/database:/app/database"]
  nginx:
    image: nginx:alpine
    ports: ["80:80", "443:443"]
    volumes: [".:/nginx.conf:/etc/nginx/nginx.conf"]
```

Opção 2: Vercel (frontend) + Railway (backend)

```
// vercel.json
{
  "framework": "nextjs",
  "buildCommand": "npm run build",
  "outputDirectory": ".next"
}
```

Opção 3: VPS completa (Nginx + PM2)

```
# deploy.sh
pm2 start backend/app/main.py --name "mv-{slug}-api"
pm2 start frontend --name "mv-{slug}-web"
nginx -s reload
```

8. ARQUIVOS A CRIAR NO PROJETO PRINCIPAL

Arquivo	Tipo	Descrição
scripts/criar-maquina-vendas.py	Script	Orquestrador principal
templates/maquina/	Diretório	Templates do projeto full-stack
templates/maquina/frontend/	Diretório	Template Next.js
templates/maquina/backend/	Diretório	Template FastAPI
templates/maquina/database/	Diretório	Schema SQL
templates/maquina/docker-compose.yml	Template	Deploy Docker
templates/maquina/vercel.json	Template	Deploy Vercel
templates/maquina/AGENTS.md	Template	Orquestrador da máquina
templates/maquina/CLAUDE.md	Template	Regras do agente
templates/maquina/SPEC.md	Template	Spec da máquina
templates/maquina/README.md	Template	Documentação
templates/maquina/.mcp.json	Template	MCPS da máquina
templates/maquina/.env.example	Template	Variáveis de ambiente
SPEC_MAQUINA_VENDAS.md	Spec	Spec do comando /criar-maquina
.claude/commands/criar-maquina.md	Command	Comando slash

9. ORDEM DE IMPLEMENTAÇÃO

Fase 1: Template Base (MVP)

1. templates/maquina/ — estrutura completa do projeto template
2. templates/maquina/frontend/ — Next.js com páginas de venda/captura
3. templates/maquina/backend/ — FastAPI com APIs de leads
4. templates/maquina/database/schema.sql — schema SQLite
5. scripts/criar-maquina-vendas.py — orquestrador

Fase 2: Subagentes e Skills

6. .claude/agents/subagente-copywriter.md (model: mimo-v2.5)
7. .claude/agents/subagente-qualificador-leads.md (model: mimo-v2.5-lite)
8. .claude/agents/subagente-analista-funil.md (model: mimo-v2.5)
9. Skills de operação (monitorar, corrigir, escalar)

Fase 3: Automação

10. scripts/lead_hunter.py — busca leads Instagram
11. scripts/email_sender.py — disparo e-mails
12. scripts/funnel_monitor.py — monitoramento 24/7

13. `scripts/auto_correct.py` — auto-correção

Fase 4: Deploy

14. `docker-compose.yml` — deploy Docker

15. `vercel.json` — deploy Vercel

16. `deploy.sh` — script de deploy

17. `README.md` — manual completo

10. VERIFICAÇÃO

Após implementação, validar: 1. `python scripts/criar-maquina-vendas.py <slug>` gera projeto completo 2. `cd marketing/maquinas/{slug}/frontend && npm install && npm run build` funciona 3. `cd marketing/maquinas/{slug}/backend && pip install -r requirements.txt && python -m uvicorn app.main:app` funciona 4. `docker-compose up` sobe frontend + backend + DB 5. Página de venda abre em `localhost:3000` 6. Formulário de captura salva lead no banco 7. API de métricas retorna dados 8. Deploy em Vercel + Railway funciona